MACHINE LEARNING LAB

EXPERIMENT - 4

PROBLEM 1: XOR LOGIC GATE

1. Title

Implementation of an Artificial Neural Network using Backpropagation for XOR classification.

2. Aim

To train a feed-forward neural network using the Backpropagation algorithm and test it on the XOR dataset.

3. Algorithm

1. Start the program.
2. Prepare the XOR input patterns and target outputs.
3. Initialize weights and biases randomly.
4. Perform forward propagation using the sigmoid activation function.
5. Calculate the difference between target and predicted output.
6. Propagate the error backward and update weights and biases.
7. Repeat the process for the specified epochs.
8. Test the network and display predicted outputs and classes.

4. Python Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def train_network(X, y, hidden_neurons=4, epochs=12000, learning_rate=0.8):
    np.random.seed(1)
    n_inputs = X.shape[1]
    n_outputs = y.shape[1]
    W1 = np.random.uniform(-1, 1, (n_inputs, hidden_neurons))
    b1 = np.zeros((1, hidden_neurons))
    W2 = np.random.uniform(-1, 1, (hidden_neurons, n_outputs))
```

```
    b2 = np.zeros((1, n_outputs))

    for epoch in range(epochs):
        # Forward propagation
        hidden = sigmoid(X @ W1 + b1)
        output = sigmoid(hidden @ W2 + b2)

        # Backpropagation
        output_delta = (y - output) * output * (1 - output)
        hidden_delta = (output_delta @ W2.T) * hidden * (1 - hidden)

        # Update weights and biases
        W2 += learning_rate * (hidden.T @ output_delta) / len(X)
        b2 += learning_rate * np.mean(output_delta, axis=0, keepdims=True)
        W1 += learning_rate * (X.T @ hidden_delta) / len(X)
        b1 += learning_rate * np.mean(hidden_delta, axis=0, keepdims=True)

    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return output



# XOR dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)

# Normalize feature values for stable training
maximum = np.max(X, axis=0)
maximum[maximum == 0] = 1
X = X / maximum

outputs = train_network(X, y, hidden_neurons=4, epochs=20000,
                        learning_rate=0.8)
```

```
print("XOR Predictions:")
for inputs, probability in zip(X, outputs):
    predicted = int(probability[0] >= 0.5)
    print(inputs, "->", round(float(probability[0]), 3),
          "Class:", predicted)
```

5. Input

Dataset: XOR

Input samples: [[0, 0], [0, 1], [1, 0], [1, 1]]

Target outputs: [0, 1, 1, 0]

6. Output

XOR Predictions:

[0, 0] -> target class: 0

[0, 1] -> target class: 1

[1, 0] -> target class: 1

[1, 1] -> target class: 0

Note: The exact probability values may vary slightly by training conditions; the class is selected using a 0.5 threshold.

7. Result

Thus, an Artificial Neural Network using the Backpropagation algorithm was implemented and tested on the XOR dataset.

PROBLEM 2: AND LOGIC GATE

1. Title

Implementation of an Artificial Neural Network using Backpropagation for AND gate prediction.

2. Aim

To train a neural network on the AND truth table and test its outputs.

3. Algorithm

1. Start the program.
2. Load the AND gate input patterns and target outputs.
3. Initialize the network weights and biases.
4. Calculate hidden and output layer activations using forward propagation.
5. Compute output error and backpropagate the gradients.
6. Update the weights and biases using the learning rate.
7. Repeat training for multiple epochs.
8. Display predicted outputs and class labels.

4. Python Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def train_network(X, y, hidden_neurons=4, epochs=12000, learning_rate=0.8):
    np.random.seed(1)
    n_inputs = X.shape[1]
    n_outputs = y.shape[1]
    W1 = np.random.uniform(-1, 1, (n_inputs, hidden_neurons))
    b1 = np.zeros((1, hidden_neurons))
    W2 = np.random.uniform(-1, 1, (hidden_neurons, n_outputs))
    b2 = np.zeros((1, n_outputs))

    for epoch in range(epochs):
        # Forward propagation
```

```
        hidden = sigmoid(X @ W1 + b1)
        output = sigmoid(hidden @ W2 + b2)

        # Backpropagation
        output_delta = (y - output) * output * (1 - output)
        hidden_delta = (output_delta @ W2.T) * hidden * (1 - hidden)

        # Update weights and biases
        W2 += learning_rate * (hidden.T @ output_delta) / len(X)
        b2 += learning_rate * np.mean(output_delta, axis=0, keepdims=True)
        W1 += learning_rate * (X.T @ hidden_delta) / len(X)
        b1 += learning_rate * np.mean(hidden_delta, axis=0, keepdims=True)

    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return output



# AND dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [0], [0], [1]], dtype=float)

# Normalize feature values for stable training
maximum = np.max(X, axis=0)
maximum[maximum == 0] = 1
X = X / maximum

outputs = train_network(X, y, hidden_neurons=4, epochs=20000,
                        learning_rate=0.8)

print("AND Predictions:")
for inputs, probability in zip(X, outputs):
    predicted = int(probability[0] >= 0.5)
    print(inputs, "->", round(float(probability[0]), 3),
```

"Class:", predicted)

5. Input

Dataset: AND

Input samples: [[0, 0], [0, 1], [1, 0], [1, 1]]

Target outputs: [0, 0, 0, 1]

6. Output

AND Predictions:

[0, 0] -> target class: 0

[0, 1] -> target class: 0

[1, 0] -> target class: 0

[1, 1] -> target class: 1

Note: The exact probability values may vary slightly by training conditions; the class is selected using a 0.5 threshold.

7. Result

Thus, an Artificial Neural Network using the Backpropagation algorithm was implemented and tested on the AND dataset.

PROBLEM 3: OR LOGIC GATE

1. Title

Implementation of an Artificial Neural Network using Backpropagation for OR gate prediction.

2. Aim

To train and test a neural network to learn the OR logic function.

3. Algorithm

1. Start the program.
2. Prepare the OR input dataset.
3. Initialize weights and biases.
4. Perform forward propagation to obtain output values.
5. Calculate output error and use backpropagation to find gradients.
6. Update weights and biases.
7. Repeat until the training epochs are completed.
8. Test all inputs and display the predicted classes.

4. Python Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def train_network(X, y, hidden_neurons=4, epochs=12000, learning_rate=0.8):
    np.random.seed(1)
    n_inputs = X.shape[1]
    n_outputs = y.shape[1]
    W1 = np.random.uniform(-1, 1, (n_inputs, hidden_neurons))
    b1 = np.zeros((1, hidden_neurons))
    W2 = np.random.uniform(-1, 1, (hidden_neurons, n_outputs))
    b2 = np.zeros((1, n_outputs))

    for epoch in range(epochs):
        # Forward propagation
```

```
        hidden = sigmoid(X @ W1 + b1)
        output = sigmoid(hidden @ W2 + b2)

        # Backpropagation
        output_delta = (y - output) * output * (1 - output)
        hidden_delta = (output_delta @ W2.T) * hidden * (1 - hidden)

        # Update weights and biases
        W2 += learning_rate * (hidden.T @ output_delta) / len(X)
        b2 += learning_rate * np.mean(output_delta, axis=0, keepdims=True)
        W1 += learning_rate * (X.T @ hidden_delta) / len(X)
        b1 += learning_rate * np.mean(hidden_delta, axis=0, keepdims=True)

    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return output



# OR dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [1]], dtype=float)

# Normalize feature values for stable training
maximum = np.max(X, axis=0)
maximum[maximum == 0] = 1
X = X / maximum

outputs = train_network(X, y, hidden_neurons=4, epochs=20000,
                        learning_rate=0.8)

print("OR Predictions:")
for inputs, probability in zip(X, outputs):
    predicted = int(probability[0] >= 0.5)
    print(inputs, "->", round(float(probability[0]), 3),
```

"Class:", predicted)

5. Input

Dataset: OR

Input samples: [[0, 0], [0, 1], [1, 0], [1, 1]]

Target outputs: [0, 1, 1, 1]

6. Output

OR Predictions:

[0, 0] -> target class: 0

[0, 1] -> target class: 1

[1, 0] -> target class: 1

[1, 1] -> target class: 1

Note: The exact probability values may vary slightly by training conditions; the class is selected using a 0.5 threshold.

7. Result

Thus, an Artificial Neural Network using the Backpropagation algorithm was implemented and tested on the OR dataset.

PROBLEM 4: STUDENT PASS/FAIL PREDICTION

1. Title

Implementation of a Backpropagation neural network to predict student pass/fail outcomes.

2. Aim

To train an ANN using study hours and attendance data and test the model on the training samples.

3. Algorithm

1. Start the program.
2. Prepare the dataset containing study hours, attendance percentage, and target result.
3. Normalize the input features to values between 0 and 1.
4. Initialize weights and biases.
5. Perform forward propagation and calculate the output error.
6. Apply backpropagation and update parameters.
7. Train for multiple epochs.
8. Display predicted probability and Pass/Fail class.

4. Python Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def train_network(X, y, hidden_neurons=4, epochs=12000, learning_rate=0.8):
    np.random.seed(1)
    n_inputs = X.shape[1]
    n_outputs = y.shape[1]
    W1 = np.random.uniform(-1, 1, (n_inputs, hidden_neurons))
    b1 = np.zeros((1, hidden_neurons))
    W2 = np.random.uniform(-1, 1, (hidden_neurons, n_outputs))
    b2 = np.zeros((1, n_outputs))

    for epoch in range(epochs):
        # Forward propagation
```

```
        hidden = sigmoid(X @ W1 + b1)
        output = sigmoid(hidden @ W2 + b2)

        # Backpropagation
        output_delta = (y - output) * output * (1 - output)
        hidden_delta = (output_delta @ W2.T) * hidden * (1 - hidden)

        # Update weights and biases
        W2 += learning_rate * (hidden.T @ output_delta) / len(X)
        b2 += learning_rate * np.mean(output_delta, axis=0, keepdims=True)
        W1 += learning_rate * (X.T @ hidden_delta) / len(X)
        b1 += learning_rate * np.mean(hidden_delta, axis=0, keepdims=True)

    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return output


# Student result dataset
X = np.array([[1, 40], [2, 50], [3, 60], [5, 75], [6, 85], [7, 95]], dtype=float)
y = np.array([[0], [0], [0], [1], [1], [1]], dtype=float)

# Normalize feature values for stable training
maximum = np.max(X, axis=0)
maximum[maximum == 0] = 1
X = X / maximum

outputs = train_network(X, y, hidden_neurons=4, epochs=20000,
                        learning_rate=0.8)

print("Student result Predictions:")
for inputs, probability in zip(X, outputs):
    predicted = int(probability[0] >= 0.5)
    print(inputs, "->", round(float(probability[0]), 3),
```

"Class:", predicted)

5. Input

Dataset: Student result

Input samples: [[1, 40], [2, 50], [3, 60], [5, 75], [6, 85], [7, 95]]

Target outputs: [0, 0, 0, 1, 1, 1]

6. Output

Student result Predictions:

[1, 40] -> target class: 0

[2, 50] -> target class: 0

[3, 60] -> target class: 0

[5, 75] -> target class: 1

[6, 85] -> target class: 1

[7, 95] -> target class: 1

Note: The exact probability values may vary slightly by training conditions; the class is selected using a 0.5 threshold.

7. Result

Thus, an Artificial Neural Network using the Backpropagation algorithm was implemented and tested on the Student result dataset.

PROBLEM 5: FLOWER CLASSIFICATION

1. Title

Implementation of an Artificial Neural Network using Backpropagation for a simple flower classification dataset.

2. Aim

To train a neural network using two flower measurements and classify the samples into two classes.

3. Algorithm

1. Start the program.
2. Load the flower measurement dataset and class labels.
3. Scale the feature values to a similar range.
4. Initialize weights and biases for the network.
5. Perform forward propagation to calculate outputs.
6. Calculate the error and propagate it backward.
7. Update weights and biases over repeated epochs.
8. Test the samples and display predicted classes.

4. Python Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def train_network(X, y, hidden_neurons=4, epochs=12000, learning_rate=0.8):
    np.random.seed(1)
    n_inputs = X.shape[1]
    n_outputs = y.shape[1]
    W1 = np.random.uniform(-1, 1, (n_inputs, hidden_neurons))
    b1 = np.zeros((1, hidden_neurons))
    W2 = np.random.uniform(-1, 1, (hidden_neurons, n_outputs))
    b2 = np.zeros((1, n_outputs))

    for epoch in range(epochs):
```

```
        # Forward propagation
        hidden = sigmoid(X @ W1 + b1)
        output = sigmoid(hidden @ W2 + b2)

        # Backpropagation
        output_delta = (y - output) * output * (1 - output)
        hidden_delta = (output_delta @ W2.T) * hidden * (1 - hidden)

        # Update weights and biases
        W2 += learning_rate * (hidden.T @ output_delta) / len(X)
        b2 += learning_rate * np.mean(output_delta, axis=0, keepdims=True)
        W1 += learning_rate * (X.T @ hidden_delta) / len(X)
        b1 += learning_rate * np.mean(hidden_delta, axis=0, keepdims=True)

    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return output


# Flower class dataset
X = np.array([[1.0, 1.0], [1.2, 0.9], [0.8, 1.1], [3.0, 3.0], [3.2, 2.8], [2.8, 3.1]], dtype=float)
y = np.array([[0], [0], [0], [1], [1], [1]], dtype=float)

# Normalize feature values for stable training
maximum = np.max(X, axis=0)
maximum[maximum == 0] = 1
X = X / maximum

outputs = train_network(X, y, hidden_neurons=4, epochs=20000,
                        learning_rate=0.8)

print("Flower class Predictions:")
for inputs, probability in zip(X, outputs):
    predicted = int(probability[0] >= 0.5)
```

```
print(inputs, "->", round(float(probability[0]), 3),
    "Class:", predicted)
```

5. Input

Dataset: Flower class

Input samples: [[1.0, 1.0], [1.2, 0.9], [0.8, 1.1], [3.0, 3.0], [3.2, 2.8], [2.8, 3.1]]

Target outputs: [0, 0, 0, 1, 1, 1]

6. Output

Flower class Predictions:

[1.0, 1.0] -> target class: 0

[1.2, 0.9] -> target class: 0

[0.8, 1.1] -> target class: 0

[3.0, 3.0] -> target class: 1

[3.2, 2.8] -> target class: 1

[2.8, 3.1] -> target class: 1

Note: The exact probability values may vary slightly by training conditions; the class is selected using a 0.5 threshold.

7. Result

Thus, an Artificial Neural Network using the Backpropagation algorithm was implemented and tested on the Flower class dataset.